

哈爾濱工業大學（深圳）

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

操作系统 (Operating System)

实验二：进程与系统调用

2026年秋季

目录



1 实验目的
2 实验任务
3 实验步骤
4 作业提交

实验目的

- 了解xv6系统调用的工作原理；
- 熟悉xv6通过系统调用给用户程序提供服务的机制；
- 理解进程和进程切换的概念；
- 了解进程的完整生命周期。

开始前的Git流程

本实验内容及要求与MIT xv6 syscall不一样，请大家以指导书为准：

1. 在做实验前需保存上一个实验的更改

- 先用**git status**确认是否修改
- 如有修改：

处理方法一：如果这些修改就是你已经完成的 Lab1 内容，请先提交：

```
git add .
git commit -m "save lab1 work"
```

处理方法二：如果你只是想先临时收起来，还不想提交，可以先暂存：

```
git stash
```

之后如果需要把这些未提交内容再拿回来，再执行：

```
git stash pop
```

处理方法三：如果你非常确定当前这些修改都不要了，也可以强制切换：

```
git checkout -f syscall
```

2. 切换到syscall分支并同步上游仓库

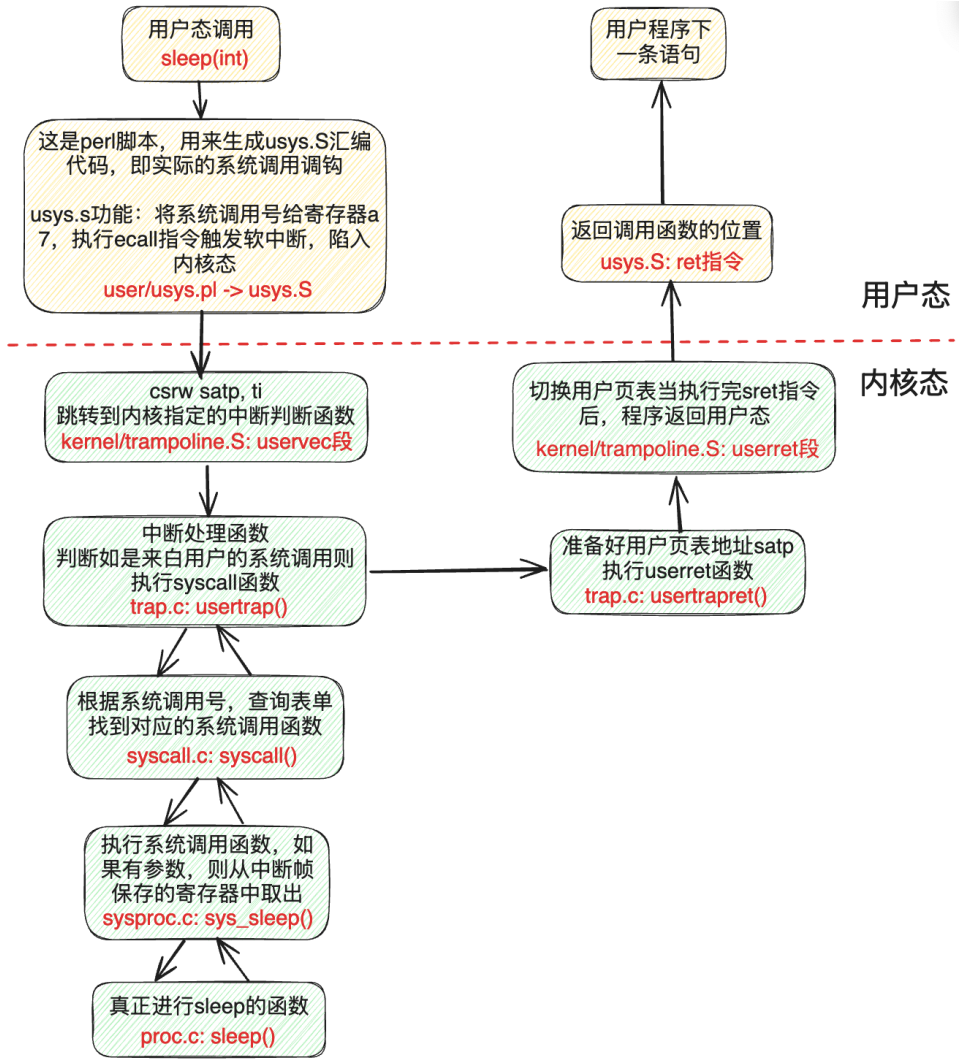
- **git remote –v** 找到指向课程实验仓库的远程名。

  多数同学是 origin；如果你的是 upstream，就改成 REMOTE=upstream。
- **REMOTE=origin**
- **git fetch origin --tags**同步远程分支和标签
- **git checkout syscall**切换到 Lab2 使用的分支
- **git pull --ff-only origin syscall**

  从远程拉取 syscall 分支的最新代码到本地
- **make clean** 清理旧编译结果和部分中间文件

实验原理 | 系统调用路径

- 新增系统调用时，要同时补齐用户态外壳和内核态分发：



- 需要看懂的四个位置

user/user.h 只是声明接口

user/usys.pl 生成 **usys.S**，写入系统调用号

trampoline.S 的 **uservec** 保存用户态寄存器

kernel/syscall.c 按 **a7**中的系统调用号分发

```
entry("yield");

li a7, SYS_yield
ecall
ret
```

实验原理 | 参数、trapframe 与 context

- Lab2 的打印信息来自两份不同的现场

- 系统调用的参数：寄存器传参

寄存器	ABI 名称	描述	调用前后是否一致?
x0	zero	硬连线为 0	—
x1	ra	返回地址	否
x2	sp	栈指针	是
x3	gp	全局指针	—
x4	tp	线程指针	—
x5	t0	临时寄存器/备用链接寄存器	否
x6–7	t1–2	临时寄存器	否
x8	s0/fp	保存寄存器/帧指针	是
x9	s1	保存寄存器	是
x10–11	a0–1	函数参数/返回值	否
x12–17	a2–7	函数参数	否
x18–27	s2–11	保存寄存器	是
x28–31	t3–6	临时寄存器	否
f0–7	ft0–7	浮点临时寄存器	否
f8–9	fs0–1	浮点保存寄存器	是
f10–11	fa0–1	浮点参数/返回值	否
f12–17	fa2–7	浮点参数	否
f18–27	fs2–11	浮点保存寄存器	是
f28–31	ft8–11	浮点临时寄存器	否

图 3.2: RISC-V 整数和浮点寄存器的汇编助记符。RISC-V 有足够的寄存器供 ABI 分配给过程自由使用，在其不产生其他调用时无需保存和恢复。过程调用前后保持不变的寄存器也称被调用者保存寄存器，反之则称调用者保存寄存器。浮点（f）寄存器将在第 5 章介绍。(此图源于 [Waterman and Asanović 2017] 的表 20.1。)

- Trapframe： **保存用户态现场**

  保存用户态寄存器状态

  a0、a1 等寄存器保存系统调用参数

  epc 保存用户态 **pc**

Context：**保存内核线程现场**

- 保存内核线程上下文
- **swtch** 会保存和恢复这份现场
- **yield** 任务要求打印 **context** 的地址范围

实验内容

任务一：进程信息收集（40分）

- 要求：在**exit**系统调用当中，使用**exit_info**打印自己的父进程和子进程的信息。

任务二：实现非阻塞方式的wait（40分）

- 要求：增加**wait**系统调用的一个**非阻塞选项参数int flags**，当**flags**为1时表示不需要进行阻塞等待，否则需要进行阻塞等待。

任务三：yield系统调用（20分）

- 要求：当调用**yield**系统调用时，功能如下：
  - ✓打印进程的内核线程**上下文被保存的地址范围**；
  - ✓打印**当前进程的pid**和此进程在用户态的**pc值**；
  - ✓打印**即将被调度到的进程的pid**和此时用户态的**pc值**；
  - ✓**将当前进程让出CPU，从而调度到别的进程**。

实验要求 | 任务一：进程信息收集

- 在exit系统调用中寻找合适的输出时间点，在相应的函数内进行父子进程信息的打印。

- **void exit(int status)** 功能：回收进程资源，回收完毕之后终止进程。
  - *当前进程的父进程*: proc *PID* exit, parent pid *PID*, name *NAME*, state *STATE*
  - *当前进程的子进程*: proc *PID* exit, child *CHILD_NUM*, pid *PID*, name *NAME*, state *STATE*
    - ✓ *PID*：进程的进程号
    - ✓ *CHILD_NUM*：该信息是第几个子进程的信息
    - ✓ *NAME*：父进程或者子进程名
    - ✓ *STATE*：父进程或者子进程的进程状态

不要使用 printf 输出
请使用 exit_info 函数输出

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ exittest
exit test
[INFO] proc 3 exit, parent pid 2, name sh, state sleep
[INFO] proc 3 exit, child 0, pid 4, name child0, state sleep
[INFO] proc 3 exit, child 1, pid 5, name child1, state sleep
[INFO] proc 3 exit, child 2, pid 6, name child2, state sleep
$ [INFO] proc 4 exit, parent pid 1, name init, state runble
[INFO] proc 5 exit, parent pid 1, name init, state run
[INFO] proc 6 exit, parent pid 1, name init, state run
```

在3号进程终止后，其所有子进程的父进程都变更为 **init** 进程

实验提示 | 任务一：进程信息收集

- （1）首先理解**user**目录下的**exittest.c**源文件（**exittest**评测命令的实现），用于验证“**父进程先退出、子进程未退出**”时的子进程状态变化。

① **main**函数调用**exittest**函数；

② **exittest**（父进程）创建 3 个命名不同的子进程（**child0**/**child1**/**child2**）；

③ 随后父进程仅睡眠 1 秒就调用**exit**退出；而 3 个子进程会睡眠 10 秒才调用**exit**退出 —— 最终会产生“孤儿进程”（父进程已死、自身未死的子进程）

```c
void exittest(void) {
  printf("exit test\n");

  for (int i = 0; i < 3; i++) {
    int pid = fork();
    if (pid == 0) {
      // 给每个子进程起一个不同的名字
      char name[16] = "child";
      char idx = '0' + i;
      int len = strlen(name);
      memmove(name + len, &idx, 1);
      name[len + 1] = '\0';
      rename(name);
      // 让子进程睡眠，使得父进程退出时子进程还活着
      sleep(10);
      exit(0);
    }
  }

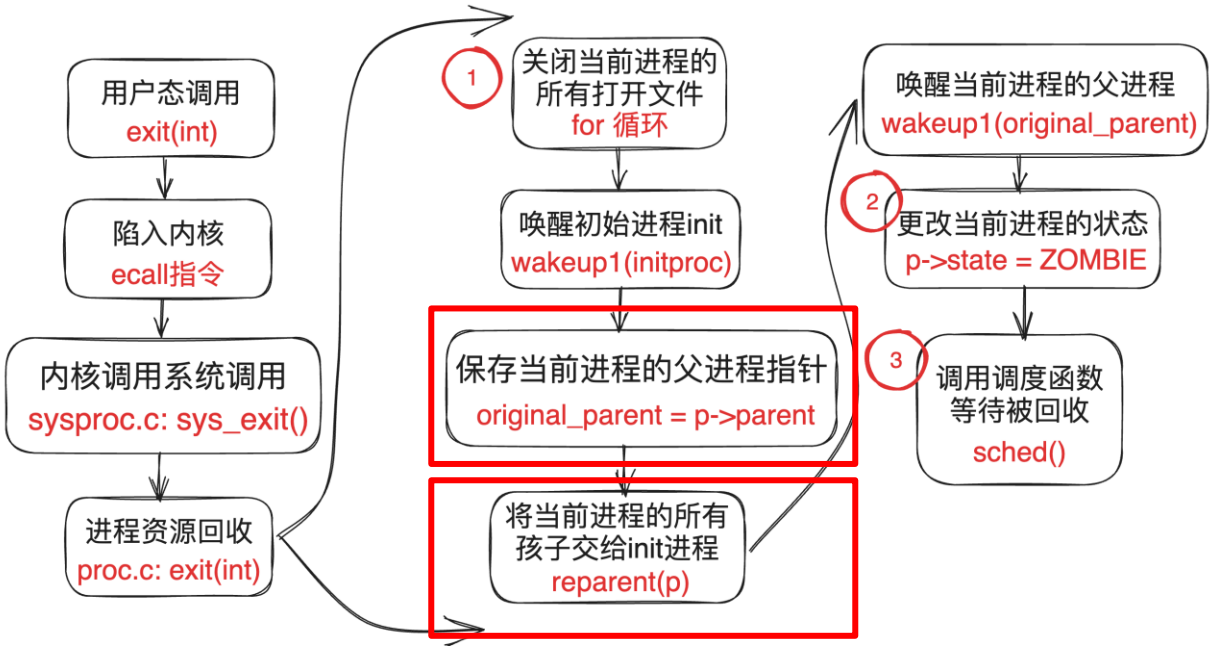
  sleep(1);
}

int main(int argc, char *argv[]) {
  exittest();
  exit(0);
}
```

实验提示 | 任务一：进程信息收集

- （2）请理解**exit**系统调用 工作流程的基础上，再实现打印信息这一功能。

- 1、如何获取父子进程信息？
  - 进程控制块（PCB）



exit系统调用工作流程

- 2、如何表征进程的状态信息？
  - 在**proc.h: enum procstate** 中选取

```
struct proc {
  struct spinlock lock;

  // p->lock must be held when using these:
  enum procstate state;        // Process state
  struct proc *parent;         // Parent process
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  int xstate;                  // Exit status to be returned to parent's wait
  int pid;                     // Process ID

  // these are private to the process, so p->lock need not be held.
  uint64 kstack;               // Virtual address of kernel stack
  uint64 sz;                   // Size of process memory (bytes)
  pagetable_t pagetable;       // User page table
  struct trapframe *trapframe; // data page for trampoline.S
  struct context context;      // swtch() here to run process
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
};
```

```
enum procstate { UNUSED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

实验要求 | 任务二：实现非阻塞方式的wait

- 在wait系统调用增加**int flags**，当**flags=1** 时不需要阻塞等待，而是直接返回**-1**

  - 非阻塞的wait：**int wait(int *status)** → **int wait(int *status, int flags)**

  - 需要改动：

    - ✓在**sys_wait** 函数中获取新添加的参数；
    - ✓更改**defs.h** 头文件的**wait**的定义；
    - ✓更改**wait** 函数以满足当前的语义；
    - ✓通过**waittest** 测试。

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ waittest
wait test
no child exited yet, round 0
no child exited yet, round 1
no child exited yet, round 2
[INFO] proc 4 exit, parent pid 3, name waittest, state sleep
child exited, pid 4
wait test OK
[INFO] proc 3 exit, parent pid 2, name sh, state sleep
$
```

实验提示 | 任务二：实现非阻塞方式的wait

- （1）首先理解user目录下的waittest.c源文件（waittest评测命令的实现），用于**测试wait 系统调用两种模式（阻塞 / 非阻塞）行为**的程序。

① 父进程前 3 次非阻塞 wait 均返回 -1，打印3条

no child exited yet， round 0
no child exited yet， round 1
no child exited yet， round 2

② 子进程 10 秒后退出，最后一次阻塞 wait 返回子进程 PID；

③ 最终打印:

child exited, pid
wait test OK

测试通过。

```
7   void waittest(void) {
8     printf("wait test\n");
9
10    int pid = fork();
11    if (pid == 0) {
12      // 这里睡眠足够长的时间，因此后面父进程进行非阻塞
    wait时该子进程一定还没有退出
13      sleep(10);
14      exit(0);
15    }
16
17    for (int i = 0; i < 3; i++) {
18      if (wait(0, nonblock) < 0) {
19        printf("no child exited yet, round %d\n", i);
20      } else {
21        printf("wait shouldn't block!\n");
22        printf("wait test FAIL\n");
23        exit(1);
24      }
25    }
26    int child_pid = wait(0, 0);
27    printf("child exited, pid %d\n", child_pid);
28
29    printf("wait test OK\n");
30  }
```

实验提示 | 任务二：实现非阻塞方式的wait

- （2）请理解**wait**系统调用 工作流程的基础上，再实现非阻塞这一功能。

➢阻塞与非阻塞实现的区别？

✓ **sleep**

➢如何获取用户态传入的参数？

✓寄存器传参

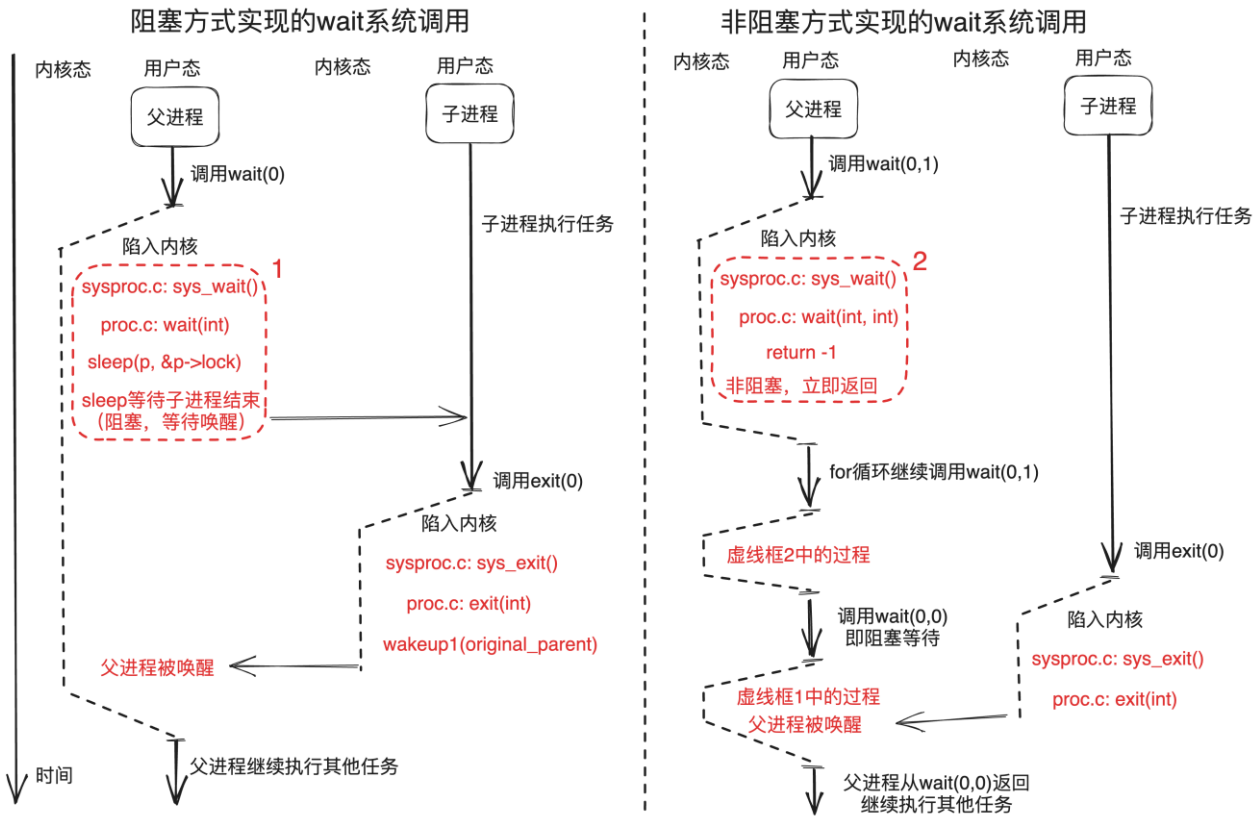
➢如何保证进程PCB的操作的互斥性？

✓ **wait函数会通过freeproc改写PCB**

✓写前上锁： **acquire(&p->lock)**

✓写后解锁： **release(&p->lock)**

✓当为非阻塞返回时，记得解锁



实验要求 | 任务三：实现yield系统调用

- 实现一个新的系统调用**yield**：

int yield()

- 具体要求如下：

```
printf("Save the context of the process to the memory region from address %p to %p\n", ?, ?);
printf("Current running process pid is %d and user pc is %p\n", ?, ?);
printf("Next runnable process pid is %d and user pc is %p\n", ?, ?);
```

  - 打印进程的内核线程上下文被保存的地址范围
  - 打印当前进程的pid和此进程在用户态的pc值
  - 打印即将被调度到的进程的pid和此时用户态的pc值
  - 将当前进程让出CPU，从而调度到别的进程

注意

- （1）需要手动将 `yieldtest` 添加进Makefile中进行编译，即找到 `UPROGS` 变量，添加一行：

```
160         $U/_zombie\
161         $U/_waittest\
162         $U/_exittest\
163         $U/_yieldtest\
164
```

- （2）该任务测试的时候需要 **设置CPU的数量为1**，即使用如下命令运行xv6:

```
make qemu CPUS=1
```

```
parent yield
Save the context of the process to the memory region from address 0x0000000080012098 to 0x0000000080012108
Current running process pid is 3 and user pc is 0x00000000000003e2
Next runnable process pid is 4 and user pc is 0x0000000000000332
Child with PID 4 begins to run
[INFO] proc 4 exit, parent pid 3, name yieldtest, state RUNNABLE
Child with PID 5 begins to run
[INFO] proc 5 exit, parent pid 3, name yieldtest, state RUNNABLE
Child with PID 6 begins to run
[INFO] proc 6 exit, parent pid 3, name yieldtest, state RUNNABLE
parent yield finished
[INFO] proc 3 exit, parent pid 2, name sh, state SLEEPING
```

实验提示 | 任务三：实现yield系统调用

- （1）首先理解**user**目录下的**yieldtest.c**源文件（**yieldtest**评测命令的实现），用于**测试进程调度中yield函数行为**的程序。在**单核处理器环境**下，该测试可直观体现yield函数**主动放弃 CPU、让其他进程优先运行**的功能。

① 父进程创建 3 个子进程后，打印**parent yield**并调用yield让出 CPU；

② 调度器切换到子进程 1，子进程 1 打印信息**Child with PID begins to run**后退出；

③ 调度器依次切换到子进程 2/3，执行相同逻辑；

④ 所有子进程退出后，父进程重新获得 CPU，打印**parent yield finished**

⑤ 父进程调用wait回收所有子进程资源，测试结束。

```
5   void yieldtest(void) {
6     printf("yield test\n");
7
8     for (int i = 0; i < 3; i++) {
9       int pid = fork();
10      if (pid == 0) {
11        printf("Child with PID %d begins to run\n", getpid());
12        exit(0);
13      } else {
14      }
15    }
16    printf("parent yield\n");
17    // yield之后
18    // 在单核情况下，调度器应当调度到子进程
19    yield();
20    printf("parent yield finished\n");
21    for (int i = 0; i < 3; i++) {
22      wait(0, 0);
23    }
24  }
```

实验提示 | 任务三：实现yield系统调用

- （2）在sys_yield函数中实现如下功能：
  - **4.** 打印即将被调度到的进程的pid和用户态pc值
    - ✓参考调度器的工作方式模拟一次调度
      - 找到下一个RUNNABLE进程
      - 增加pid和pc值的打印信息
    - ✓记得注意锁的获取和释放
  - **5.** 将当前进程让出CPU，从而调度到别的进程
    - ✓调用yield()函数（XV6内核态已实现）

scheduler 与 swtch

调度切换的是寄存器现场，不是复制进程对象

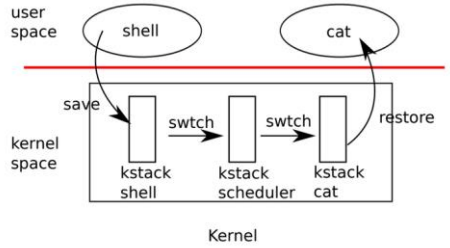


Figure 7.1: Switching from one user process to another. In this example, xv6 runs with one CPU (and thus one scheduler thread).

```
void scheduler(void) {
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for (;;) {
    // Avoid deadlock by ensuring that devices can interrupt.
    intr_on();

    int found = 0;
    for (p = proc; p < &proc[NPROC]; p++) {
      acquire(&p->lock);
      if (p->state == RUNNABLE) {
        // Switch to chosen process.  It is the process's job
        // to release its lock and then reacquire it
        // before jumping back to us.
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context); //切换到p进程
        // p进程yield或者时间片用完回到调度器线程继续执行下一行代码
        // Process is done running for now.
        // It should have changed its p->state before coming back.
        c->proc = 0;

        found = 1;
      }
      release(&p->lock);
    }
    if (found == 0) {
      intr_on();
      // wfi(Wait for Interrupt)是一个特定的低功耗汇编指令，使 CPU 进入节能状态，直到下一次中断发生
      asm volatile("wfi");
    }
  }
}
```

实验步骤 | 实验测试

- 当在命令行输入 **make grade** 进行测试。如果通过测试，会显示如下内容：

- 只有做了附加题，才需要再跑 **make grade-extra**

```
$ make qemu-gdb LAB_SYSCALL_TEST=on
exit test: OK (4.0s)
== Test wait test ==
$ make qemu-gdb LAB_SYSCALL_TEST=on
wait test: OK (2.0s)
== Test yield test ==
$ make qemu-gdb CPUS=1 LAB_SYSCALL_TEST=on
yield test: OK (0.5s)
== Test time ==
time: OK
Score: 100/100
```

常见错误定位

先看错误发生在哪一层，再决定查用户程序还是内核路径

启动后提示 /init 不存在
- 通常不是 scheduler 这一行本身错了
- 先检查文件系统镜像是否包含 init
- 确认 make clean 后重新构建

yield 没有输出或无法链接
- 检查 user/usys.pl，而不是手改 usys.S
- 检查 syscall.h 的系统调用号
- 检查 syscall.c 的 extern 和 syscalls[]
- 检查 Makefile 是否加入 _yieldtest

wait 非阻塞分支
flags == 1 时提前返回，必须先释放相关锁，否则后面的进程管理逻辑可能卡住。

yield 输出顺序
手动测试使用 make qemu CPUS=1。多核环境下不同 CPU 同时输出，结果更容易乱序。

实验提交

■ 请务必注意查看作业提交平台，按时提交代码及报告

- ✓ 实验二提交截止时间：两个星期内
  - ✓ 实验报告：回答问题，并写清三项基础任务的设计思路与结果
  - ✓ **commit.patch文件**

工作分支是 syscall

自检时，要求使用标签 syscall-26-fall-v2

Hope you enjoyed the OS course!